

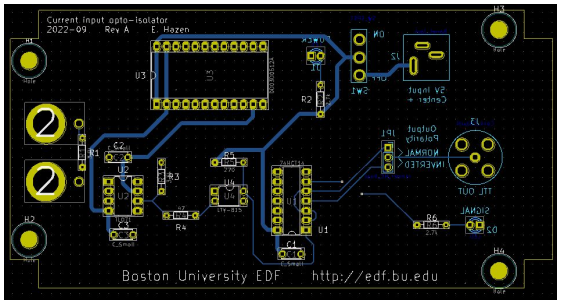
BOSTON UNIVERSITY

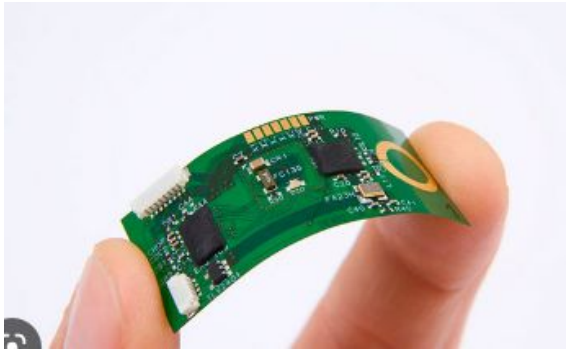


Git, GitHub and KiCAD

Mini-Tutorial focused on KiCAD PCB design

Eric Hazen
(Sr. Research Engineer, BME)





What is this presentation?

It is:

- A very quick introduction to Git and GitHub with a focus on KiCAD

It is NOT:

- A detailed explanation of Git

At the end, you should be able to work on a KiCAD design in Github.

For *a lot* of interesting detail on how Git works, see this from MIT:

https://missing.csail.mit.edu/2026/version-control/

From xkcd.com



About version control and Git



A version control system, or VCS, tracks the history of changes as people and teams collaborate on projects together.
As developers make changes to the project, any earlier version of the project can be recovered at any time.

Developers can review project history to find out:

- Which changes were made?
- Who made the changes?
- When were the changes made?
- Why were changes needed?

GitHub is a Microsoft cloud-based storage system where you can store *repositories*.
Each *repository* contains a collection of related files/folders.

For now, use one repository for each of your KiCAD designs.

Getting started with Git



- Create a **GitHub** account. Know your username (maybe use the same username as BU if it's available)

  Access at e.g. https://github.com/eshazen

- Install Git client software from https://git-scm.com/

  - Windows, MacOS, download and install as usual
  - Linux (Ubuntu): `sudo apt install git`

- Initial setup steps (next slide)
  Needed on each computer you want to use

Initial setup (once on each computer)

- ***Use the command-line*** (sorry, if you want to do it graphically you're on your own)
  Windows: open **powershell** (or git-bash)
  Mac, Linux: open a **terminal**

First time only:

```
git --version
```
should display a version# to confirm git is installed correctly

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
```

```
ssh-keygen
```
(press Enter several times)

This last step generates an "ssh key" which allows you to use Git without authenticating each time. Depending on the ssh version you *might* have to use the long form below:

```
ssh-keygen -t ed25519 -C "your_email@example.com"
```

In either case, don't enter any passkey, just press "Enter

Add ssh key to your github account

(to avoid typing your password every time you commit)

Log on to github.com .
Click profile picture at the top right.
Click “settings” at the top and “SSH and GPG keys” at the left. Click “New SSH Key”

Now you need to copy/paste the contents of the file `$HOME/.ssh/id_ed25519.pub` into the “Key”

In any operating system this should work from the command line:

```
cat $HOME/.ssh/id_ed25519.pub
```

Select the entire text string starting with “ssh-ed25519” with the mouse and copy (Ctrl-C or apple-C)
Paste into the dialog box on github.com
Type something descriptive into the “Title” box and click “Add SSH Key”.

Use https://github.com/eshazen/kicad-template

Using this Template in KiCad 10.0

Download KiCad here. I currently recommend version 10.

1. Click the `Use this Template` to make a personal copy of this repository.
2. Set the repository name to your desired name: PROJECT_NAME
3. Clone the repository to your local machine
4. Open up KiCad 10.0 and open the default project in the `kicad/template` directory.
5. Click `save as` and save it to the `kicad` directory under your desired name PROJECT_NAME.
6. The folder tree should look like this:

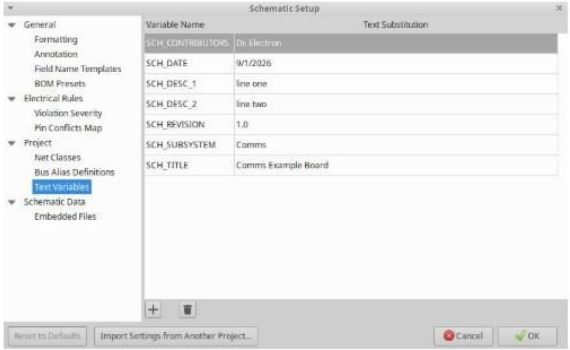
```
PROJECT_NAME/
├── kicad/
│   ├── template/
│   │   ├── template.kicad_dru
│   │   ├── template.kicad_pcb
│   │   ├── template.kicad_pro
│   │   ├── template.kicad_sch
│   │   └── sheet.kicad_wks
│   └── PROJECT_NAME/
│       ├── PROJECT_NAME.kicad_dru
│       ├── PROJECT_NAME.kicad_pcb
│       ├── PROJECT_NAME.kicad_pro
│       ├── PROJECT_NAME.kicad_sch
│       └── sheet.kicad_wks
├── datasheets
├── calculations
├── img
├── README.md
└── LICENSE
```

7. Delete the `kicad/template` directory.

Using this template allows you to get started quickly with a KiCAD with many useful default settings.

Instructions at the link…

8. Open up your newly renamed project, `PROJECT_NAME.kicad_pro`, and then open the schematic editor.
9. Click `File -> Schematic Setup`, then under `Project -> Text Variables`, populate all of the fields with the correct information such as `contributors` and board `revision`. Then click OK and you should see the bottom right of the schematic and layout correctly update. See a screenshots below:

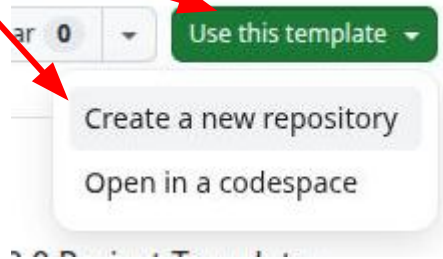


8. Edit this README.md file, initialize the git repository, and start working on your project!

Using my KiCAD template…

Navigate to: https://github.com/eshazen/kicad-template

Click



Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? Import a repository.
Required fields are marked with an asterisk (*).

Start with a template
Templates pre-configure your repository with files.
eshazen/kicad-template

Include all branches
If enabled, all branches from the template repository will be included.
Off

1 **General**

Owner * eshazen / **Repository name *** hazen-be511-lab

hazen-be511-lab is available.

Great repository names are short and memorable. How about fantastic-train?

Description
My lab project - Arduino shield
31 / 350 characters

2 **Configuration**

Choose visibility *
Choose who can see and commit to this repository
Public

Fill in the dialog box.
Include your name in the repo name.

Clone the (empty) repository on your computer

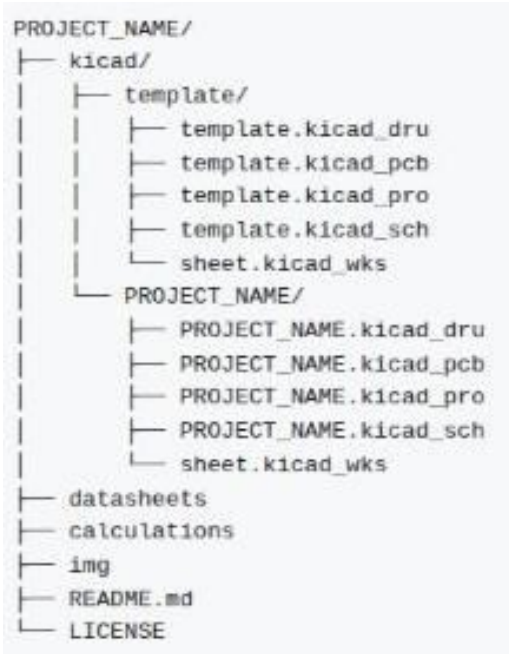
(open a terminal / shell)

```
cd ~                    make sure you are in your home folder
git clone git@github.com: username/repo-name
```

Open KiCAD and open the default project in the `kicad/template` directory.

Click save as and save it to the kicad directory using your desired project name.

Open your newly renamed project in KiCAD
And then open the schematic editor.
You're ready to start a schematic!



Editing your design



Make sure KiCAD is not running!

Before starting - open terminal and navigate to your repo folder

```
cd ~/repo-name        (or as appropriate)
git pull              (retrieve any changes from github)
```

Open KiCAD and work as usual, *in the folder where you cloned the repo, e.g.*

```
$HOME/repo-name
```

When you are done editing (or at least once a day or more often):

Exit KiCAD (always do this before pushing to Git)

Open terminal and navigate to your repo folder.
Maybe "`cd ~/repo-name`"

```
git add -A                                                  (add all modified files)
git commit -m "please write a short description"            (commit changes to local repo)
git push                                                    (push changes to github)
```

Collaborating with others



Each team member should create a github account
Add each team member as a collaborator (settings→collaborators) on web page

In general, only one person should edit at a time, and ***commit*** and ***push*** their work to git before anyone else touches it.

Some hints / observations



- KiCAD 10 has built-in Git integration. Don’t use it! There are bugs…
- Don’t use *branches* or *tags* unless everyone on the team understands them.
- If you get stuck with e.g. merge errors, clone the repo to a new place and if necessary copy edited files to the new clone and push to Git.
- *Commit* and *push often!* You can use git’s history features to get back to any previous state where you did a commit.
- If you lose your computer or delete the design, you can easily clone the repo again on yours or another computer